Ann Oper Res (2007) 150:115–135
DOI 10.1007/s10479-006-0158-9

A market-based multi-agent system model for decentralized multi-project scheduling

Giuseppe Confessore · Stefano Giordani · Silvia Rismondo

Published online: 21 December 2006
© Springer Science + Business Media, LLC 2007

Abstract We consider a multi-project scheduling problem, where each project is composed of a set of activities, with precedence relations, requiring specific amounts of local and shared (among projects) resources. The aim is to complete all the project activities, satisfying precedence and resource constraints, and minimizing each project schedule length. The decision making process is supposed to be decentralized, with as many local decision makers as the projects. A multi-agent system model, and an iterative combinatorial auction mechanism for the agent coordination are proposed. We provide a dynamic programming formulation for the combinatorial auction problem, and heuristic algorithms for both the combinatorial auction and the bidding process. An experimental analysis on the whole multi-agent system model is discussed.

Keywords Multi-project scheduling · Multi-agent system · Combinatorial auction · Heuristic algorithms

Introduction

A multi-project environment is quite common in modern day industry. Indeed, companies often manage various projects simultaneously sharing a set of resources. Moreover, due to the rapid growth of the Internet technology and globalization, project environments are becoming more distributed both geographically and organizationally, implying an informational and a managerial decentralization. In this paper, we consider a decentralized multi-project scheduling problem consisting of a set of projects, each one composed of a set of activities, with precedence relations, requiring specific amounts of local and shared (among projects)

G. Confessore (✉) · S. Rismondo
Istituto di Tecnologie Industriali e Automazione, Consiglio Nazionale delle Ricerche,
Area della Ricerca di Roma Tor Vergata, Via del Fosso del Cavaliere 100, 00133 Roma, Italy
e-mail: g.confessore@itia.cnr.it

S. Giordani
Dipartimento di Ingegneria dell'Impresa, Università di Roma "Tor Vergata"

renewable resources. The aim of the multi-project scheduling problem is to complete all the activities of the projects, satisfying precedence and limited resource availability constraints, and minimizing the length of each project schedule. The decision process is supposed to be decentralized, with as many local and autonomous decision makers (project managers) as the projects.

If the activities of each project require only local resources, we can decompose the whole multi-project problem into a set of independent (single) project scheduling problems (with a single decision maker) and solve them independently. On the contrary, when activities of different projects require a shared resource, possible conflicts arise among such activities; in this case, the local decision making processes are no longer independent, and a coordination mechanism between them is necessary to solve possible conflicts and allocate the shared resource to projects.

We model the decentralized multi-project scheduling problem as a multi-agent system, in which multiple agents—corresponding to the local decision makers (project managers)—collaborate, guided by a market-based control mechanism which is coordinated by a (virtual) coordinator agent. Within this architecture, the project managers mutually compete for the shared resource in order to complete their projects as soon as possible, and the coordinator is entrusted for solving shared resource allocation conflicts among activities belonging to different projects.

While there are many papers in the open literature addressing the resource-constrained (single) project scheduling problem (RCPSP)—which is known to be strongly *NP*-hard—(e.g., see the survey of Brucker et al. (1999)), less attention has been devoted to the multi-project case. Moreover, the majority of the papers addressing the multi-project scheduling problem considers the centralized problem version where all the projects are managed by a single manager (e.g., see Lova and Tormos, 2001). Recently, a distributed (decentralized) multi-project scheduling approach based on multi-agent system has been proposed by Lee, Kumara, and Chatterjee (2003) for short-term rescheduling of resources shared by multiple projects. Other examples of multi-agent system approaches in the scheduling field have also been given by Agnetis et al. (2001), Arbib and Rossi (2000), and Knotts, Dror, and Hartman (2000).

The market-oriented control approach we propose for resolving the conflicts between projects and coordinate the agents is based on an iterative combinatorial auction process. At each round of the auction process, each losing bidder among the bidding agents (i.e., the project managers) makes a new bid for a set of shared resource time slots (i.e., a bundle of goods), in response to the bids made by the winning bidders. In particular, this new bid is made for the bundle that maximize the bidder's utility, given the current bundle prices, which is determined by solving a bidding problem. Once all the bids have been collected, the auctioneer (i.e., the coordinator) finds a provisional allocation of bundles of goods to the (new) winning bidders, that maximizes his/her revenue. At the end of each round, the auctioneer updates the bundle prices, which will become valid for the next round, and communicates the new prices to bidders as well. The auction process ends when all the bidding agents are winners.

Since both the combinatorial auction problem and the bidding problem are hard to be solved exactly and have to be solved several times in the iterative auction process, we provide and use heuristic algorithms for both these problems. In particular, the heuristics for the former problem are based on a relaxation of a dynamic programming formulation that we propose for the combinatorial auction problem.

A wide experimental analysis of the performance of the whole market-based multi-agent system modeling framework—where all the provided heuristic algorithms are integrated—on

a set of multi-project instances is described. Besides, some experiments are also discussed for evaluating the performance of the heuristic algorithms for the combinatorial auction problem.

The remainder of the paper is organized as follows. In Section 1, we formally define the decentralized multi-project scheduling problem. In Section 2, we model the decentralized decision process as a multi-agent system and the agent coordination mechanism as an iterative combinatorial auction process, describing in detail all the sub-processes involved. In Section 3, we provide a simple example of this mechanism. Section 4 is devoted to the performance analysis of the whole multi-agent model. Finally the Appendix contains the details of the dynamic programming formulation and relaxation for the combinatorial auction problem, along with the discussion of the experimental analysis of the performance of the proposed heuristic algorithms.

1 The decentralized multi-project scheduling problem

The multi-project scheduling problem we consider is defined as follows. A set P of z projects is given. Each project i of P consists of n_i activities, where each activity has to be executed without preemption in order to complete the project. Finish-start precedence relations among activities of the same project i are given. For each project i, a set R_i of s_i *local* renewable resources is given, with resource h_i of R_i being available in ρ_{h_i} units. Renewable means that a pre-specified number of units of a resource is available for every unit time period of the planning horizon. Moreover, a *shared* single unit renewable resource $\bar{h}$, which can be used by any activity of any project, is also given. Each activity j_i of project i requires $r_{j_i h_i}$ units of resource $h_i \in R_i$, with $0 \leq r_{j_i h_i} \leq \rho_{h_i}$, and $r_{j_i \bar{h}} \in \{0, 1\}$ units of shared resource $\bar{h}$, for its whole duration d_{j_i}. The parameters d_{j_i}, $r_{j_i h_i}$ and ρ_{h_i} are assumed to be non-negative integers. The activities are interrelated by two kinds of constraints: precedence constraints forcing an activity not to be started before all its predecessors have been finished; resource constraints due to the limited resource availability in each unit time period. The objective of the problem is to find a precedence and resource feasible schedule for all the activities of each project, such that the schedule length (i.e., the makespan) of each project is minimized.

We suppose that the decision making process is decentralized, with as many local and autonomous decision makers (project managers) as the projects. We refer to this problem as the *Decentralized Resource Constrained Multi-Project Scheduling Problem* (DRCMPSP).

2 Modeling the decentralized decision process

We model the DRCMPSP as a multi-agent system, that is, a distributed system composed of a set of self-contained problem-solving entities (e.g., see Jennings and Wooldridge, 1995) called agents. Each agent operates collecting data from the environment, elaborating them, and applying strategies in order to achieve its goal. Moreover, each agent has incomplete information, there is not a centralized system control, the computation are asynchronous, and data are decentralized. In fact, in a multi-agent system we could have logically, and geographically spread information.

Two key issues in a multi-agent system are the classes of agents, and the communication protocol, that is, the set of rules used to exchange messages. Each message is an information that the agents use to pursue their goal. Concerning with the first issue, we consider two types of agents dealing with different decisional problems, respectively: (1) the project managers, who have to schedule their project activities requiring the shared resource in specific time

slots; (2) a coordinator agent, who is responsible for allocating the shared resource time slots to project managers, and, hence, for solving shared resource conflicts among projects.

The coordination process supervised by the coordinator agent is modeled by a market oriented approach, that is, by means of an iterative auction process (e.g., see Klemperer, 1999; McAfee and McMillan, 1987). Typically, an auction process concerns three different typologies of actors: the sellers, the buyers, and the auctioneer. Without loss of generality, we will further assume that all the goods belong to the coordinator agent (i.e., the auctioneer), we do not consider any sellers, and the bidders—acting as the buyers—are the project managers.

Recently market oriented approaches like auctions have been applied in solving scheduling problems. Wellman et al. (2001) considered a subclass of single machine scheduling problems, where each agent has one job to complete. In that case, they showed that any optimal solution to the single-unit scheduling problem (i.e., when job durations are equal to one) is supported by a price equilibrium, that is, given the current goods prices, each job agent gets a good (i.e., the single machine in a time slot) maximizing its utility. Among the different auction typologies, they obtained interesting theoretic results by using ascending auction for the single-unit case, while they showed that for the multiple-unit scheduling problem (i.e., with job durations greater than one), the ascending auction can produce allocations that are arbitrarily far from the optimal solution. In light of these drawbacks, several researchers have proposed combinatorial auction models, where agents submit offers for a combination of goods. For instance, Sandholm (2000) showed that, when bundles of items are considered, the combinatorial auction overcomes the economical inefficiencies of sequential and parallel auctions.

Accordingly, we consider a combinatorial auction since in our case the project managers require the shared resource in a set of time slots, that is, a bundle of goods. In particular, we consider an iterative ascending price bundle combinatorial auction (e.g., see Parkes, 1999). Figure 1 depicts the functional schema of the multi-agent system representing the interaction between $z + 1$ agents in the iterative auction process (agent 0 is the auctioneer, and agent i, for $i = 1, 2, \ldots, z$, is one of the bidders), where the rectangular blocks represent local decision processes, and the dashed block refers to the auctioneer one.

At each round of the iterative combinatorial auction process, each bidding agent (i.e., project manager) i formulates its bid given by the pair $\langle price, bundle_i \rangle$, where *price* is the cost of the required $bundle_i$.

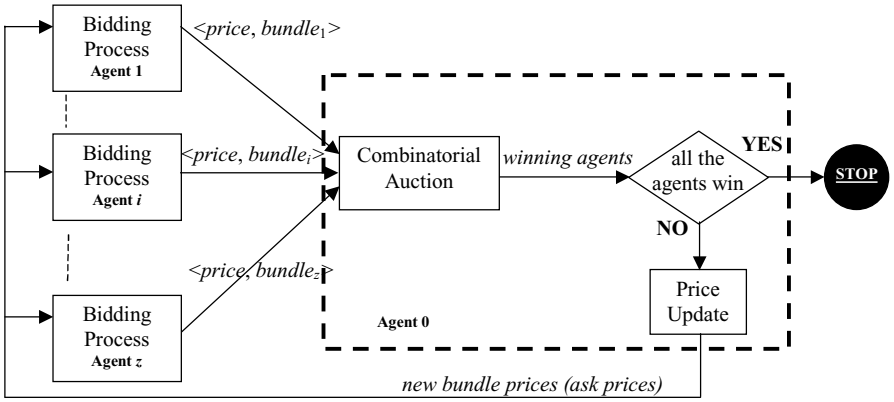


Fig. 1 Multi-agent system functional schema

The auctioneer collects all the bidders' bids, and finds a provisional allocation of bundles to winning bidders, maximizing his/her selling revenue (see Section 2.1). The bidders receiving the provisional allocation of the bid bundles are the *winners*, while the others are the *losers*. At the end of each round, the auctioneer updates the bundle prices (see Section 2.2), and communicates to the bidders these new prices (i.e., the *ask prices*) for the bundles which will be valid for the next round. Supposing the bidding agents to be myopic best-response, in the next round, the winning bidders (in the previous round) will bid for the same bundle at the same price, while each losing bidder will evaluate the bundles with the new prices and formulate a new bid for the bundle that maximize his/her utility function. The bundle utility function depends on the project schedule length with the shared resource availability restricted to the time slots of the bundle, and the best bundle is determined by solving the bidding problem defined in Section 2.3. The iterative combinatorial auction process ends when all the bidders are winners, that is all the bid bundles are allocated.

In the following subsections, we explain in detail the main processes of the proposed iterative combinatorial auction model.

2.1 The combinatorial auction problem

Let $\{i : i = 1, \ldots, z\}$ be the set of bidders participating to the iterative auction process; we recall that the bidder i, with $i = 1, \ldots, z$, is the agent representing the project manager of project i. Let T be an upper bound on the schedule length of a project (e.g., T is the sum of the durations of all the multi-project activities requiring the shared resource plus the maximum, among the projects, of the total duration of the other activities of each project). Let $\Gamma_T = \{1, \ldots, T\}$ be the set of single-unit goods, that is, the set of time slots where the shared resource is available.

At each round k, each bidder submits a single bid for a bundle of goods (i.e., a subset of Γ_T in which he/she can employ the shared resource to execute the activities of the project he/she manages). Let the pair $\langle p_i^k, S_i^k \rangle$ be the bid of bidder i at round k, where $p_i^k \geq 0$ is the price of the bundle $S_i^k \subseteq \Gamma_T$ required by bidder i, and let $\lambda_{i\tau}^k \in \{0, 1\}$, for $\tau = 1, \ldots, T$, be equal to 1 if good τ belongs to bundle S_i^k, and 0 otherwise.

The auctioneer receives a bid from each bidder and has to declare which are the winners, provisionally allocating to them the sets of single-unit goods (bundles) they wish to purchase. The auctioneer problem is to label all the bids as winning or losing in order to maximize the auctioneer's revenue subject to the constraints that no more than one unit of good τ, with $\tau = 1, \ldots, T$, is allocated to the bidders. The goods are distinguishable, and we consider the free disposal case in which the seller (the auctioneer in our case) does not need to sell everything. Let x_i^k be a binary variable associated to agent i at round k, being equal to 1 if the bid of bidder i is labeled as winner, and 0 otherwise. As in Sandholm et al. (2002), the problem can be formulated as follows:

$$w^*(k) = \max \sum_{i=1}^{z} p_i^k \cdot x_i^k$$

$$\text{s.t.} \sum_{i=1}^{z} \lambda_{i\tau}^k \cdot x_i^k \leq 1, \quad \tau = 1, \ldots, T,$$

$$x_i^k \in \{0, 1\}, \quad i = 1, \ldots, z.$$

The combinatorial auction problem is known to be *NP*-hard also in the special case of single-unit goods we consider (Rothkopf, Pereč, and Harstad, 1998).

Since this problem has to be solved at each round of the iterative auction process, we solve it heuristically by means of the following fast heuristic algorithms based on a relaxation (DPR) of a Dynamic Programming (DP) formulation of the combinatorial auction problem. Both the DP and DPR formulations are described in the Appendix.

Given an ordered list of the bidders, all the proposed heuristic algorithms (called H1, H2, H3, and H4) construct a feasible solution by considering the bidders one at time, following the order in which they are in the list, and add to the solution the bid of the bidder if his/her required bundle is a subset of yet available (free) goods.

The proposed algorithms adopt different ordering criteria. As for algorithms H1 and H2, the list L of bidders is $L = (L_W, L_L)$, with L_W containing the winning bidders in the DPR solution, and L_L containing the losers. For H1, both L_W, and L_L are ordered by non-increasing ratios $(p_i^k/|S_i^k|)$ between the bidding price and the total number of time slots in the required bundle. For H2, both L_W, and L_L are ordered by non-increasing bidding price.

As for algorithms H3 and H4, we do not consider any information from the DPR solution, and order the list L by non-increasing ratios $(p_i^k/|S_i^k|)$ between the bidding price and the total number of time slots in the required bundle, and by non-increasing bidding prices, respectively.

An analysis of the performance of these algorithms is discussed in the Appendix.

2.2 The price updating

In this subsection, we explain how the auctioneer defines the new price (i.e., the *ask price*) $p^{k+1}(S)$ of a generic bundle S of goods valid for the next round $k+1$. In general, the *ask price* $p^{k+1}(S)$ is a lower bound on acceptable bid prices, that should allow a losing bidder at round k to possibly win the bundle S at round $k+1$. Clearly, the *ask price* of a bundle cannot be smaller than the maximum ask price of bundles that it contains.

Let L^k be the set of losing bidders and let $p^k(S)$ be the price of a generic bundle S at round k. For each bundle S, we consider a *reserve price* $\bar{p}$ as well, and, hence, $p^1(S) = \bar{p}$ is its starting price; in particular, we do not allow reserve price discrimination, and, hence, we consider the same reserve price for all the bundles (e.g., $\bar{p} = 1$).

To obtain $p^{k+1}(S)$, first we compute the price $\pi^{k+1}(S)$, as the value $p^k(S)$ plus the minimum bid increment ε, if at round k there is at least one losing bidder that bids for bundle S and hence allowing that bidder to possibly win S at round $k+1$; otherwise, we set $\pi^{k+1}(S) = p^k(S)$. Formally, we have:

$$\pi^{k+1}(S) = p^k(S) + \varepsilon^k(S), \quad \text{with } \varepsilon^k(S) = \begin{cases} \varepsilon & \text{if } \left\{i \in L^k : S_i^k = S\right\} \neq \emptyset \\ 0 & \text{otherwise.} \end{cases}$$

In order to enforce the ask price $p^{k+1}(S)$ of bundle S to be equal to the maximum ask price of the bundles it contains, we finally consider $p^{k+1}(S) = \max[\pi^{k+1}(S), \max_{S' \subset S}\{\pi^{k+1}(S')\}]$.

2.3 The bidding process

At round $k+1$, with $k > 0$, while winning bidders bid as in round k (i.e., for the same bundle at the same price), the losing bidders in L^k (at round k) have to formulate a new bid for a

bundle with the bid price being equal to the *ask price* for that bundle; if $k = 0$ (i.e., at round 1) all the bidders have to formulate their first bid, hence, $L^0 = \{1, \ldots, z\}$.

The bidder i in L^k selects a bundle S to bid on the basis of the utility value that he/she ascribes to that bundle. In particular, at round $k+1$ bidder i makes a new bid for the bundle $\bar{S}_i^{k+1}$ that maximizes the bidder's local utility function $u_i^{k+1}(S)$. The utility value $u_i^{k+1}(S)$ ascribes by bidder i to bundle S at round $k+1$ is defined as the difference between the value $v_i(S)$ that bidder i ascribes to that bundle, and the price $p^{k+1}(S)$ of S at round $k+1$. That is, $u_i^{k+1}(S) = v_i(S) - p^{k+1}(S)$.

We consider $v_i(S)$ as a linear decreasing function of the ratio between $\delta_i(S)$ and l_i, where l_i is the minimum length (duration) of project i supposing that the shared resource is always available (i.e., removing shared resource availability constraints), and $\delta_i(S)$ is the increment of the project length with respect to l_i, when the shared resource is available only in the set S of time slots; that is, $\delta_i(S) = l_i(S) - l_i$, where $l_i(S)$ is the minimum project length of project i when the shared resource is available only in the time slots belonging to S.

In particular, $v_i(S)$ is as follows:

$$v_i(S) = v_i^{\max} - K_i \frac{\delta_i(S)}{l_i}.$$

The parameters $v_i^{\max}$ and K_i concern the bidding agent behavior in the bidding process: $v_i^{\max}$ is the maximum bundle value among all the possible bundles for bidder i, that is, the value that bidder i ascribes to bundle S when $\delta_i(S) = l_i(S) - l_i = 0$; K_i represents the aggressiveness of bidder i.

Neglecting the constant $v_i^{\max}$, the problem of finding the bundle $\bar{S}_i^{k+1}$ of maximum utility for bidder i in L^k at round $k+1$, called the *bidding problem*, can be reformulated as

$$\min_{S \subseteq \Gamma_T} K_i \frac{\delta_i(S)}{l_i} + p^{k+1}(S).$$

Note that, not always we have to solve the bidding problem. Let $\bar{S}_i^k$ and $\tilde{S}_i^k$ be, respectively, the best and the second best bundles for bidder i in L^k at round k, and let $\Delta_i^k = u_i^k(\bar{S}_i^k) - u_i^k(\tilde{S}_i^k)$ be the utility gap between the first and the second best possible choices. We recall that we are considering an ascending (iterative) combinatorial auction, and, hence, $p^{k+1}(\bar{S}_i^k) - p^k(\bar{S}_i^k) \geq 0$.

If $p^{k+1}(\bar{S}_i^k) - p^k(\bar{S}_i^k) \leq \Delta_i^k$ then the best bundle at round k for bidder i is still the best at round $k+1$ (i.e., $\bar{S}_i^{k+1} = \bar{S}_i^k$) ; note that, if the price $p^{k+1}(\tilde{S}_i^k) \neq p^k(\tilde{S}_i^k)$, in general, we have that $\tilde{S}_i^{k+1} \neq \tilde{S}_i^k$, and, hence, we need to find the second best bundle $\tilde{S}_i^{k+1}$.

Vice-versa, if $p^{k+1}(\bar{S}_i^k) - p^k(\bar{S}_i^k) > \Delta_i^k$, we need to compute the best bundle $\bar{S}_i^{k+1}$, and, moreover, if the (new) best bundle $\bar{S}_i^{k+1}$ is equal to $\tilde{S}_i^k$ or $p^{k+1}(\tilde{S}_i^k) \neq p^k(\tilde{S}_i^k)$ also the second best bundle $\tilde{S}_i^{k+1}$ has to be found.

Finding the (new) best bundle $\bar{S}_i^{k+1}$ (and/or the second best bundle $\tilde{S}_i^{k+1}$) is a strongly *NP*-hard problem. Indeed, even computing $\delta_i(S) = l_i(S) - l_i$ for a given bundle S is strongly NP-hard, since we have to compute $l_i(S)$, that is, to solve an instance of the RCPSP (which is known to be strongly *NP*-hard) with time-varying resource availability constraint (e.g., see Kubiak et al., 2002) on the shared resource (which is considered to be available only in the set S of time slots).

Therefore, when it is necessary to determining the best and/or the second best bundles, which in the worst case it is required for each bidder i and for each round $k+1$, we make use of the following fast heuristic algorithm for the bidding problem. We search the best and

the second best bundles among the best and the second best bundles $\bar{S}_i^{\lambda}$, $\tilde{S}_i^{\lambda}$ at the previous rounds $\lambda = 1, \ldots, k$, and the set $\mathcal{B}_i^{k+1}$ of (new) bundles contained in the set $\Gamma_T \backslash \{1, \ldots, \bar{t}\}$ of time slots, with $\bar{t} = \max[\max_{\lambda=1,\ldots,k}\{\bar{t}_i^{\lambda}\}; \max_{\lambda=1,\ldots,k}\{\tilde{t}_i^{\lambda}\}]$ where $\bar{t}_i^{\lambda}$ and $\tilde{t}_i^{\lambda}$ are the first time slot in $\bar{S}_i^{\lambda}$ and $\tilde{S}_i^{\lambda}$, respectively; if $k = 0$, we set $\bar{t} = 0$, and search only among the bundles in $\mathcal{B}_i^1$ contained in the set Γ_T of time slots.

In particular, since $\delta_i(S) = l_i(S) - l_i$ is a non-decreasing function of activity ending times, and supposing that the prices of the bundles in $\mathcal{B}_i^{k+1}$ are the same and equal to the reserve price $\bar{p}$, the best bundle $\bar{B}_i^{k+1}$ and second best bundle $\tilde{B}_i^{k+1}$ among the bundles in $\mathcal{B}_i^{k+1}$ are obtained as follows. As for $\bar{B}_i^{k+1}$, we solve the (single-project) RCPSP instance of project i supposing that the shared resource $\bar{h}$ is not available in the set of time slots $\{1, \ldots, \bar{t}\}$; then, according to the computed RCPSP solution, we set $\bar{B}_i^{k+1}$ equal to the set of time slots in which resource $\bar{h}$ is required by the scheduled activities of project i. As for $\tilde{B}_i^{k+1}$, we solve the RCPSP instance of project i with $\bar{h}$ being non-unavailable in the set of time slots $\{1, \ldots, \bar{t}^+\}$, with $\bar{t}^+$ being the first time slot in $\bar{B}_i^{k+1}$, and we set $\tilde{B}_i^{k+1}$ equal to the set of time slots in which $\bar{h}$ is required.

Finally, for solving the RCPSP with time-varying resource availability constraint, we make use of a fast heuristic algorithm. Since providing a new heuristic algorithm for this problem is out of the scope of this paper, we adopt the well known heuristic algorithm based on the parallel generation schema with the *latest finish time* priority rule, that was experimentally shown to be the most effective among all the priority rule based heuristic algorithm (Kolish, 1996). Generally, a priority rule based algorithm is made up of two components, a scheduling generation schema and a priority rule.

Here is a synthetic description of the parallel generation schema (for the details, the reader may refer to Kolish, 1996). The parallel generation schema generates a feasible schedule by extending a partial schedule (i.e., a schedule where only a subset of the activities are scheduled) in a stage-wise fashion. Given a schedule time τ, the parallel schema maintains three subsets of the activities: the *complete set* (i.e., the scheduled activities which are finished), the *active set* (i.e., the scheduled activities which are not finished), and the *decision set* (i.e., the eligible subset of the unscheduled activities). In each stage, the following step are made: (1) the schedule time τ equal to the earliest finish time of the activities in the active set is computed; (2) all the activities in the active set that are completed at the schedule time τ are removed and added to the complete set; (3) the decision set, that is, the set of unscheduled activities which are available for scheduling w.r.t. precedence and resource constraints, is defined; (4) a specific priority rule is then employed in order to choose one activity from the decision set; this activity is removed from the decision set and added to the active set. The algorithm terminates when all activities are in the complete or active sets.

3 Multi-agent system mechanism example

In this section, we describe a simple example explaining the whole multi-agent system decisional process. We consider a multi-project environment with $z = 3$ different projects composed of $n = 5$ activities (where two of them are dummies). We consider two local renewable resources for each project (i.e., $s_i = 2$, for each project i, with $i = 1, 2, 3$) and a shared one. We suppose that only one activity per project can require the shared resource for its execution. We set the auction parameters as follows: $\varepsilon = 10$; $\bar{p} = 1$; $v_i^{\max} = K_i^{\max} = 13$ for all $i = 1, 2, 3$.

Figures 2–4 represent the structure of project $i = 1, 2$, and 3, respectively. Each figure shows the activity-on-node network of project i, and the label $[d_{j_i}, (r_{j_i 1_i}, r_{j_i 2_i}, r_{j_i \bar{h}})]$ of each

Fig. 2 Project 1

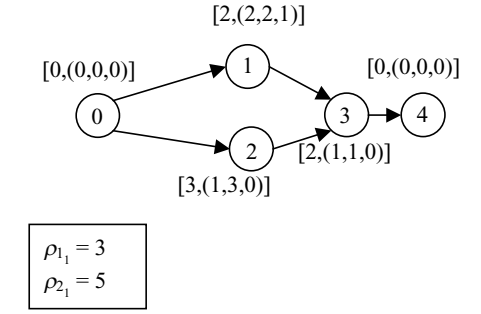


Fig. 3 Project 2

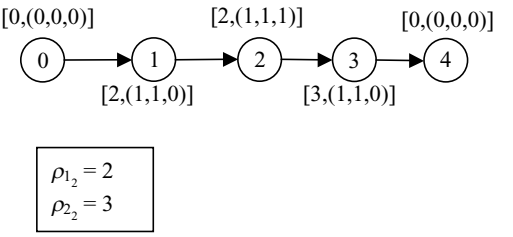
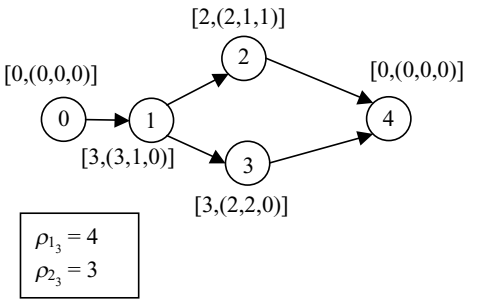


Fig. 4 Project 3



node reports the duration and resource requirement of activity j_i represented by the node; the available units ρ_{1_i} and ρ_{2_i} of the local resources 1_i and 2_i are specified inside the box, while the shared resource availability $\rho_{\bar{h}}$ is equal to 1.

In the following, we describe the decisional process.

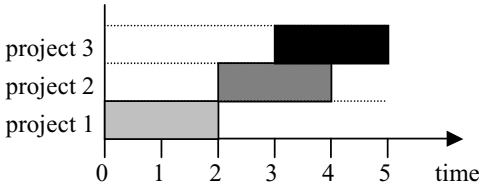
Round 1

– *Price updating*

The auctioneer opens the auction process announcing a reserve price $\bar{p}$ for each bundle.

– *Bidding process*

Fig. 5 Round 1: bid bundles



All the agents have to formulate a bid, hence, they have to compute their best and second best bundle:

- Project 1: Best bundle $\bar{S}_1^1 = \{1, 2\}$, with the minimum project length $l_1(\bar{S}_1^1) = 5$, and utility $u_1^1(\bar{S}_1^1) = 12$; second best bundle $\tilde{S}_1^1 = \{2, 3\}$, with $l_1(\tilde{S}_1^1) = 5$ and $u_1^1(\tilde{S}_1^1) = 12$;
- Project 2: Best bundle $\bar{S}_2^1 = \{3, 4\}$, with $l_2(\bar{S}_2^1) = 7$ and $u_2^1(\bar{S}_2^1) = 12$; second best bundle $\tilde{S}_2^1 = \{4, 5\}$, with $l_2(\tilde{S}_2^1) = 8$ and $u_2^1(\tilde{S}_2^1) = 10.14$;
- Project 3: Best bundle $\bar{S}_3^1 = \{4, 5\}$, with $l_3(\tilde{S}_3^1) = 6$ and $u_3^1(\tilde{S}_3^1) = 12$: second best bundle $\tilde{S}_3^1 = \{5, 6\}$, with $l_3(\tilde{S}_3^1) = 6$ and $u_3^1(\tilde{S}_3^1) = 12$.

Note that for projects 1 (and for project 3) the activity requiring the shared resource does not lies on the critical path, and then the unitary right shift of its starting time (necessary to compute the second best bundle for the project), and consequently the unitary right shift of its shared resource requirement, does not impact on the length of the project. Vice-versa, for project 2 the unitary right shift of the starting time of the activity requiring the shared resource increases the project length, and, consequently, the utility of the second best bundle is less than the utility of the best bundle.

Figure 5 represents the shared renewable resource requirements of the three projects according to their best bundles. Each bidder i will bid for its best bundle $\bar{S}_i^1$ at the reserve price.

– *Combinatorial auction process*

The auctioneer declares bidders 1 and 2 as the winners and bidder 3 as the loser. The auctioneer's revenue is equal to 2. Since not all the agents are winners the iterative process continues.

Round 2

– *Price updating*

The auctioneer increases the price of bundle $\bar{S}_3^1$ that was bid by the losing bidder 3, since only this agent was not a winner. The new ask price will be $p^2(\bar{S}_3^1) = 11$.

– *Bidding process*

Bidder 1 and 2 do not modify their bid since they won at the previous round. Bidder 3 has to modify his/her bid. Since Δ_3^2 is equal to 0 (i.e., 12 – 12), and $p^2(\bar{S}_3^1) - p^1(\bar{S}_3^1) > \Delta_3^2$, then bidder 3 has to compute the new best bundle $\bar{S}_3^2$. Since, the (new) best bundle $\bar{S}_3^2$ is equal to $\tilde{S}_3^1 = \{5, 6\}$, then the second best bundle $\tilde{S}_3^2$ has to be found. Bidder 3 computes the new second best bundle by searching among the best bundles $\bar{S}_3^\lambda$ and the second best bundle $\tilde{S}_3^\lambda$ at the previous round ($\lambda = 1$), and among the set $\mathcal{B}_3^2$ of new bundles. The bundle $\bar{S}_3^1 = \{4, 5\}$, at the current price $p^2(\bar{S}_3^1) = 11$, has the utility value $u_3^2(\bar{S}_3^1) = 2$, and, hence, less than the utility of the new bundle $\{6, 7\} \in \mathcal{B}_3^2$, which at the (current) reserve price, is

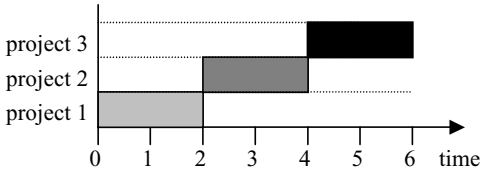


Fig. 6 Round 2: bid bundles

$u_3^2(\{6,7\}) = 9.83$. Therefore, bidder 3 sets the new second best bundle $\tilde{S}_3^2 = \{6,7\}$, with $l_3(\tilde{S}_3^2) = 7$ and $u_3(\tilde{S}_3^2) = 9.83$. Finally, bidder 3 will bid for $\bar{S}_3^2 = \{5,6\}$ at the (current) reserve price.

Figure 6 represents the shared renewable resource requirements of the three projects according to their best bundles. Each bidder i will bid for its best bundle $\bar{S}_i^2$ at the current price.

– *Combinatorial auction process*

The auctioneer declares bidders 1, 2, and 3 as winning bidders. The auctioneer's revenue is equal to 3. All the agents are winners then the iterative process ends.

Figure 6 shows also the final shared renewable resource allocation to the projects. With this resource allocation, project 1 has (minimum) length equal to 5, project 2 has 7, and project 3 has length equal to 6. Note that, for all the projects, these are the minimum project lengths also when the shared resource is always available to all the projects.

Although this example is clearly not exhaustive, the goal of this section is to explain step-by-step the whole decisional process. To conclude this section we point out that, having centralized information, the simple observation of the activities lying on the critical path of the projects can produce in one step the same solution of the multi-agent system model.

4 Experimental analysis

All the heuristic algorithms have been integrated in the multi-agent system framework, have been coded in C language and run on a 2.53 GHz Pentium 4 PC.

Next, we analyze the performance of the multi-agents system model by comparing the results given by the four heuristic algorithms (i.e., H1, H2, H3, and H4) used to solve the combinatorial auction.

We have experimented with a set of randomly generated multi-project instances with $z = 3$ or 5 projects, with each project composed of $n = 10$ or 18 non-preemptive activities. In this way, small-sized problem instances have 30 activities and large-sized problem instances have 90 activities. Each project i, for $i = 1, \ldots, z$, has been randomly generated accordingly to the project scheduling instance generation method by Kolisch and Sprecher (1996). Finish-start precedence relations among activities of the same project i are represented by the activity-on-node project network, with the network complexity, that is, the ratio between the number of direct (non-redundant) precedence relations to the number of activities, being set to 3. The resource set R_i is composed of $s_i = 2$ local renewable resources with fixed integer availability ρ_{h_i} of resource $h_i \in R_i$ drawn from the interval [1, 10]. The shared (among all the projects) renewable resource $\bar{h}$ has a fixed availability equal to 1.

Each activity j_i of project i has a fixed integer duration d_{j_i} drawn from the interval [1, 10]. The resource factor, which indicates the average number of local resources used by each activity, is on average equal to 1.3; the requirement of each positive resource demand $r_{j_i h_i}$ is

between 1 and ρ_{h_i}. Finally, instance parameter m corresponding to the maximum number of activities per project requiring the shared renewable resource $\bar{h}$ is set to $m = 1$, 2 or 3. For each combination of (z, n, m) parameters we have generated 20 different instances.

As for the instance parameters related to the multi-agent market-based coordination mechanism modeled by the proposed iterative combinatorial auction process, we consider the minimum bid increment $\varepsilon = 1$, 2 or 3, and the bidder aggressiveness factor K_i is drawn from the interval [20, 50]. For the sake of completeness, we also set the bundle reserve price $\bar{p}$ to 1, and the maximum bundle value $v_i^{\max}$ to 10 for all the bidders, even if the model, and, hence, the results are independent from the values of these two latter parameters.

The performances of the multi-agents system model are evaluated by computing the following indexes:

- the *average project delay 'APD'* with respect to the minimum project length (duration) when no shared resource constraints are considered. That is, $APD = \sum_{i=1}^{z}(l_i^{\text{end}} - l_i)/z$, where l_i is the minimum project length of project i supposing that the shared resource is always available, and l_i^{end} is the length of project i at the end of the iterative auction.
- the *average relative gap 'ARG %'* (in percentage) between the length of the project with and without shared resource (i.e., $ARG\,\% = 100 \times ((\sum_{i=1}^{z} l_i^{\text{end}} - \sum_{i=1}^{z} l_i)/\sum_{i=1}^{z} l_i)$).

Figures 7 and 8 summarize the heuristic algorithms behavior by showing the range of the *APD* values for each one of the heuristics, respectively for $z = 3$, and $z = 5$. The x-axis refers to the heuristic algorithm, while the y-axis indicates the *APD* value. In particular, the boxes refer to the minimum, the average, and the maximum *APD* values.

For $z = 3$ projects, Fig. 7 shows that there are slight differences in the *APD* values, with algorithms H2 and H4 performing favorably with respect to the maximum and the average

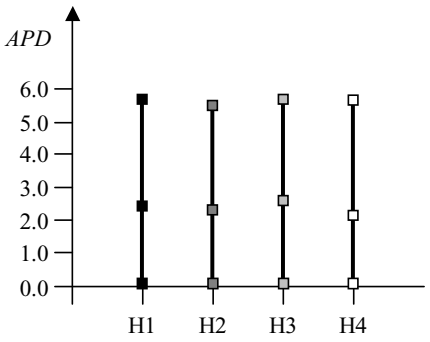


Fig. 7 Heuristics comparison ($z = 3$)

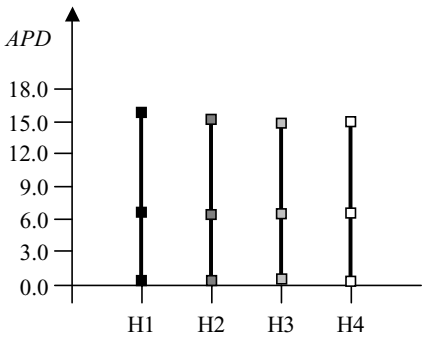


Fig. 8 Heuristics comparison ($z = 5$)

Table 1 Experimental results: (H1, $z = 3$).

ε	n	m	Ave. #-rounds	*APD*	*ARG %*	CPU time
1	10	1	5.40	0.13	0.5	0.13
1	10	2	58.00	1.93	7.9	1.82
1	10	3	334.60	3.20	7.7	15.88
2	10	1	4.00	0.13	0.5	0.07
2	10	2	33.60	1.93	7.9	0.94
2	10	3	175.20	2.67	6.7	5.72
3	10	1	4.40	0.13	0.5	0.08
3	10	2	24.60	1.93	7.9	0.69
3	10	3	109.40	2.60	6.6	3.35
1	18	1	14.80	0.73	1.8	0.40
1	18	2	387.80	5.27	13.2	16.95
1	18	3	79.00	3.87	8.0	3.04
2	18	1	11.40	0.73	1.8	0.27
2	18	2	208.60	5.87	14.6	6.66
2	18	3	44.80	4.07	8.4	1.35
3	18	1	9.80	0.73	1.8	0.22
3	18	2	140.20	5.40	13.4	4.44
3	18	3	45.00	4.27	8.8	1.38

values, respectively. For $z = 5$ (see Fig. 8), the minimum and average *APD* values are almost the same with all the four algorithms, while the maximum *APD* values show that algorithm H3 performs better. Clearly, by increasing the number of projects, that is by increasing the number of conflicts between the activities requiring the shared resource, we observe an increasing of the *APD* values.

The whole detailed results are given in Tables 1–8. Tables 1–4 show the experimental results obtained for $z = 3$ by using the heuristic algorithms for the combinatorial auction, while Tables 5–8 contains the results with z =5. The reported values are the average over 20 test problems with the same parameter values. In each table, columns one, two and three report the minimum bid increment ε, the number n of activities per project, and the maximum number m of activities per project requiring the shared resource $\bar{h}$, respectively. Column four lists the average number of rounds ('Ave. #-rounds') of the iterative combinatorial auction. Column five represents the average project delay *'APD'*. Column six reports the average relative gap '*ARG%*' (in percentage). Finally, in column seven the computational time in seconds ('CPU time') is reported.

Evaluating the *APD* performance with respect to the minimal bid increment ε, we may observe that there are no significant changes with respect to this parameter. Moreover, analyzing the average *APD* values obtained for fixed z and n parameter values we can see that heuristic H2, and H4 provide the best average *APD* values for $z = 3$, $n = 10$, while H4 provides the best average *APD* value for $z = 3$, $n = 18$. For $z = 5$ projects and $n = 10$ activities H2 and H3 give the best average *APD* values, while for $z = 5$ and $n = 18$, H4 always provides the best average results.

The 'Ave. #-rounds' value increases when the number n of project activities increases, and clearly we observe the same behavior increasing the number z of projects. The minimum average number of rounds for $z = 3$ is 93.92, and it is obtained with H1, while for $z = 5$ the minimum average value is equal to 239.14 (with H2). Moreover, the 'Ave. #-rounds' value decreases by increasing the minimal bid increment ε. In fact, with larger ε values the

Table 2 Experimental results: (H2, $z = 3$)

ε	n	m	Ave. #-rounds	*APD*	*ARG %*	CPU time
1	10	1	5.40	0.13	0.5	0.13
1	10	2	73.80	2.13	8.8	2.15
1	10	3	303.20	2.53	6.4	9.85
2	10	1	4.00	0.13	0.5	0.07
2	10	2	46.00	2.00	8.2	1.37
2	10	3	162.60	2.67	6.7	5.17
3	10	1	4.40	0.13	0.5	0.09
3	10	2	38.80	2.00	8.2	1.12
3	10	3	110.40	2.60	6.6	3.34
1	18	1	14.80	0.73	1.8	0.36
1	18	2	357.60	4.87	12.3	12.85
1	18	3	141.40	3.80	8.0	4.12
2	18	1	11.40	0.73	1.8	0.27
2	18	2	189.00	5.53	13.9	6.48
2	18	3	87.80	4.33	9.5	2.91
3	18	1	9.80	0.73	1.8	0.22
3	18	2	129.20	4.93	12.4	4.27
3	18	3	79.00	4.67	10.0	2.53

Table 3 Experimental results: (H3, $z = 3$)

ε	n	m	Ave. #-rounds	*APD*	*ARG %*	CPU time
1	10	1	27.40	0.13	0.5	0.75
1	10	2	74.00	2.13	8.7	2.04
1	10	3	259.00	2.67	6.4	10.51
2	10	1	6.00	0.13	0.5	0.13
2	10	2	41.80	2.13	8.7	1.04
2	10	3	142.40	2.67	6.4	4.19
3	10	1	6.00	0.13	0.5	0.12
3	10	2	31.40	2.13	8.7	0.77
3	10	3	99.60	2.67	6.4	2.83
1	18	1	9.80	0.47	1.2	0.24
1	18	2	337.60	5.40	13.7	13.75
1	18	3	194.20	4.93	10.5	7.94
2	18	1	5.60	0.73	1.8	0.12
2	18	2	188.40	5.40	13.7	5.52
2	18	3	104.80	4.93	10.5	2.95
3	18	1	4.40	0.73	1.8	0.07
3	18	2	133.40	5.40	13.7	3.74
3	18	3	74.80	5.87	12.3	2.05

prices for the already evaluated bundles increase quickly (and consequently the utility quickly decreases), forcing the bidding agents to explore more frequently new bundles.

The computational time 'CPU Time' behavior shows the same trend of the average rounds with a minimum average value of 3.07 seconds for $z = 3$ (with H4), and a minimum average value of 11.58 seconds for $z = 5$ (with H2).

Table 4 Experimental results: (H4, $z = 3$)

ε	n	m	Ave. #-rounds	*APD*	*ARG %*	CPU time
1	10	1	5.40	0.13	0.5	0.10
1	10	2	67.40	2.00	8.2	2.00
1	10	3	306.20	2.47	6.3	9.19
2	10	1	4.00	0.13	0.5	0.06
2	10	2	43.80	2.07	8.5	1.27
2	10	3	168.60	2.53	6.4	4.92
3	10	1	4.40	0.13	0.5	0.08
3	10	2	32.80	2.07	8.5	0.88
3	10	3	116.80	2.53	6.4	3.38
1	18	1	11.00	0.73	1.8	0.24
1	18	2	430.60	4.93	12.5	13.15
1	18	3	146.60	3.67	7.7	4.54
2	18	1	7.00	0.73	1.8	0.14
2	18	2	218.60	5.67	14.2	6.29
2	18	3	88.80	3.27	7.1	2.63
3	18	1	5.00	0.73	1.8	0.08
3	18	2	141.00	5.07	12.8	4.03
3	18	3	78.40	3.47	7.6	2.27

Table 5 Experimental results: (H1, $z = 5$)

ε	n	m	Ave. #-rounds	*APD*	*ARG %*	CPU time
1	10	1	75.20	0.76	2.7	2.64
1	10	2	655.80	8.84	36.6	39.55
1	10	3	701.80	9.36	34.7	31.93
2	10	1	54.40	1.20	4.5	1.67
2	10	2	443.60	9.20	38.1	21.31
2	10	3	376.60	10.12	37.3	15.87
3	10	1	46.60	1.36	5.2	1.44
3	10	2	294.00	9.60	39.9	13.83
3	10	3	225.60	9.12	33.6	9.59
1	18	1	21.80	0.56	1.4	0.72
1	18	2	288.60	4.72	10.6	16.23
1	18	3	641.60	14.80	33.9	34.77
2	18	1	30.40	1.00	2.5	1.04
2	18	2	151.60	4.68	10.6	8.03
2	18	3	362.80	15.00	34.3	18.95
3	18	1	27.80	1.04	2.6	0.86
3	18	2	115.40	4.56	10.3	5.60
3	18	3	359.60	16.04	36.7	18.40

A further investigation has been done for comparing the performance of the decentralized model with respect to a centralized one. In order to make this comparison we have compared the minimum *APD* values obtained by the (decentralized) multi-agent system model architecture with the four combinatorial auction heuristic algorithms with the *ADP* values obtained with the centralized heuristic approach described in Lova and Tormos (2001). Referring to that paper, we apply one of the best priority rule-based heuristic algorithm for the average

Table 6 Experimental results: (H2, $z = 5$)

ε	n	m	Ave. #-rounds	*APD*	*ARG %*	CPU time
1	10	1	75.20	0.76	2.7	2.41
1	10	2	701.20	8.76	36.2	32.82
1	10	3	556.20	9.08	33.5	22.99
2	10	1	54.40	1.20	4.5	1.71
2	10	2	360.60	8.88	36.8	16.56
2	10	3	360.80	9.44	35.0	14.97
3	10	1	46.60	1.36	5.2	1.47
3	10	2	280.00	9.28	38.5	12.94
3	10	3	223.00	9.12	33.8	9.25
1	18	1	21.80	0.56	1.4	0.72
1	18	2	193.80	3.80	8.4	11.01
1	18	3	639.20	15.08	34.5	38.01
2	18	1	30.40	1.00	2.5	1.01
2	18	2	114.40	3.56	7.9	6.28
2	18	3	272.60	14.52	33.2	15.87
3	18	1	27.80	1.04	2.6	0.87
3	18	2	104.00	4.92	11.2	5.47
3	18	3	242.60	15.20	34.8	14.09

Table 7 Experimental results: (H3, $z = 5$)

ε	n	m	Ave. #-rounds	*APD*	*ARG %*	CPU time
1	10	1	59.20	0.72	2.7	1.84
1	10	2	947.00	9.24	38.2	45.50
1	10	3	591.80	8.84	32.8	29.18
2	10	1	36.40	0.80	3.1	0.97
2	10	2	441.40	9.44	38.9	16.59
2	10	3	347.20	9.08	33.8	12.52
3	10	1	28.40	0.80	3.1	0.76
3	10	2	300.60	9.20	37.9	11.52
3	10	3	245.20	9.04	33.6	8.87
1	18	1	49.40	1.64	4.3	1.67
1	18	2	303.40	4.40	9.9	16.93
1	18	3	635.00	14.84	33.9	42.97
2	18	1	28.40	1.60	4.2	0.81
2	18	2	154.00	4.52	10.2	5.75
2	18	3	293.60	14.52	33.2	12.33
3	18	1	23.60	1.60	4.2	0.70
3	18	2	111.80	4.52	10.2	4.45
3	18	3	214.80	14.52	33.2	9.50

project delay (*APD*) minimization, i.e., the parallel schema heuristic with SASP (Shortest Activity from Shortest Project) priority rule. The SASP rule allows to identify the activity having the minimum g_{ij_i} value among the activities of all the projects, where g_{ij_i} is equal to the initial duration of project i (obtained with the CPM analysis) plus the duration of activity j_i of project i.

The results for the comparison are reported in Table 9.

Table 8 Experimental results: (H4, $z = 5$)

ε	n	m	Ave. #-rounds	*APD*	*ARG %*	CPU time
1	10	1	96.80	1.20	4.6	2.78
1	10	2	718.00	9.76	39.8	31.25
1	10	3	638.60	9.64	36.1	28.74
2	10	1	47.00	1.00	4.0	1.33
2	10	2	390.40	9.56	39.2	16.77
2	10	3	411.20	10.12	37.4	17.47
3	10	1	38.80	1.12	4.4	1.12
3	10	2	268.40	9.40	38.8	11.53
3	10	3	320.60	10.56	39.8	13.82
1	18	1	18.00	0.48	1.2	0.55
1	18	2	250.80	3.56	7.9	11.68
1	18	3	731.80	15.04	34.4	35.63
2	18	1	14.80	0.68	1.7	0.39
2	18	2	144.20	3.84	8.6	6.65
2	18	3	433.40	15.00	34.3	21.13
3	18	1	14.60	0.56	1.4	0.38
3	18	2	104.60	3.56	7.9	4.62
3	18	3	278.00	14.88	34.0	13.14

Table 9 Decentralized vs. centralized approach comparison

z	n	m	*APD* Dec.	*ARG %* Dec.	*APD* Cen.	*ARG %* Cen.
3	10	1	0.13	0.5	1.13	4.3
		2	1.93	7.9	3.20	12.8
		3	2.47	6.3	2.27	5.2
3	18	1	0.47	1.2	0.47	1.2
		2	4.87	12.3	3.47	8.6
		3	3.27	7.1	4.00	8.3
5	10	1	0.72	2.7	1.13	4.3
		2	8.76	36.2	3.20	12.8
		3	8.84	32.8	2.27	5.2
5	18	1	0.48	1.2	0.47	1.2
		2	3.56	7.9	3.47	7.6
		3	14.52	33.2	5.00	10.4

Columns one, two, and three of this table list the number z of projects, the number n of activities per project, and the parameter m, respectively. Columns four and five, named '*APD* Dec.' and '*ARG %* Dec.', list the *APD* values and the *ARG %* (in percentage) values, respectively, obtained by our decentralized multi-agent system model. Columns six and seven, named '*APD* Cen.' and '*ARG %* Cen.', list the *APD* values and the *ARG %* (in percentage) values, respectively, obtained by the centralized approach of Lova and Tormos (2001). From the table we may notice that for $z = 3$ the two compared models have almost the same performance, and in some cases our decentralized multi-agent system model outperforms the centralized approach. For $z = 5$, with the exception of the case with $n = 10$ and $m = 1$, the centralized approach performs better. On the other hand, two aspects have to be take into account when making this comparison. First the multi-agent system model does not directly optimize the index *APD*, and second it is not possible to use the centralized approach when

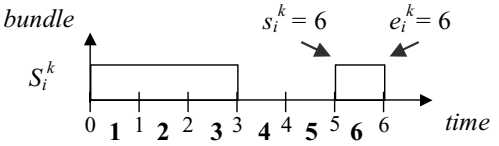


Fig. 9 An example of a bundle S_i^k

the information is not centralized as it is assumed through this work. Within this respect, the comparison allows to quantify the impact of the lack of information sharing among the project organizations in terms of performance worsening.

5 Conclusions

In this paper we have considered a multi-project scheduling problem, where each project is composed of a set of activities, with precedence relations, requiring specific amounts of local and shared resources. The aim is to complete all the project activities, satisfying precedence and resource constraints, and minimizing each project schedule length. The decision making process is decentralized, with as many local decision makers as the projects.

We proposed a multi-agent system model for the considered problem, in which multiple agents compete, guided by a market-based control mechanism which is coordinated by a coordinator agent. We provided the details of the market-based protocol based on a combinatorial auction.

The analysis results support the idea that the proposed approach can be directly applied to many decentralized scheduling problems, and to other decentralized resource allocation problems arising in supply chains or in industrial districts. We believe that the combination of the agent-based approaches, and the traditional scheduling techniques could be promising particularly when the information is not centralized, and can not be applied traditional (scheduling) techniques.

Future works will be devoted to test the proposed approach on real cases, and provide better heuristic algorithms for the decision processes involved in the proposed multi-agent system architecture. Moreover, an interesting problem is to adapt the multi-agent system when either there are more than one shared resource and/or the shared resource availability is greater than one.

Appendix

- Combinatorial auction problem: Dynamic Programming formulation and relaxation

Next, similarly to Sandholm and Suri (2000), we propose a Dynamic Programming (DP) formulation for the combinatorial auction problem.

Let the bundle S_i^k, required by bidder i at round k, be the union of two disjoint subsets Y_i^k and X_i^k, where $X_i^k = \{s_i^k, \ldots, e_i^k\}$ is the last set of consecutive time slots in S_i^k. As an example, in Fig. 9 bidder i requires the bundle S_i^k, with $Y_i^k = \{1, 2, 3\}$ and $X_i^k = \{6\}$.

Let $f^k(t, \bar{\Gamma}_t)$ be the optimal solution value of the combinatorial auction problem at round k restricted to the set $\Gamma_t = \{1, \ldots, t\}$ of the first t time slots, and with $\bar{\Gamma}_t \subseteq \Gamma_t$ being the subset of non-allocated (free) time slots in Γ_t. The value of $f^k(t, \bar{\Gamma}_t)$ is defined by the following

recursion:

(DP)

$$f^k(t,\bar{\Gamma}_t)=\begin{cases} f^k(t-1,\bar{\Gamma}_t\setminus\{t\}) & \text{if } t\in\bar{\Gamma}_t,\\ \max\left[-\infty;\ \max\limits_{\{i:e_i^k=t,\,S_i^k\cap\bar{\Gamma}_t=\emptyset\}}\left\{f^k\left(s_i^k-1,\bar{\Gamma}_t\cup Y_i^k\right)+p_i^k\right\}\right] & \text{otherwise}\end{cases}$$

with $f^k(0,\emptyset)=0$.

The optimal solution value $w^*(k)$ of the combinatorial auction problem is:

$$w^*(k)=\max_{\Gamma_T\subseteq\Gamma_T} f^k(T,\bar{\Gamma}_T)$$

Clearly, the computational time required for computing $w^*(k)$ is an exponential function of the number of time slots, since for each stage t of the DP recursion, we need to compute the value $f^k(t,\bar{\Gamma}_t)$ for all the pairs (states) $(t,\bar{\Gamma}_t)$, which are as many as the number of the subset of $\bar{\Gamma}_t$. Nevertheless, we can efficiently compute an upper bound $UB(k)$ on the optimal solution value $w^*(k)$, by a Dynamic Programming Relaxation (DPR) based on the state space relaxation technique. Let $\bar{f}^k(t,\theta)$ be the optimal solution value of the combinatorial auction problem at round k, considering the first t time slots, with $\theta\le t$ being the number of non-allocated (free) time slots in the set Γ_t. The recursion expression of the DPR is:

(DPR)

$$\bar{f}^k(t,\theta)=\max\left[-\infty;\ \bar{f}^k(t-1,\theta-1);\ \max_{\{i:e_i^k=t,\,\theta+|Y_i^k|\le s_i^k-1\}}\left\{\bar{f}^k\left(s_i^k-1,\theta+\left|Y_i^k\right|\right)+p_i^k\right\}\right],$$

with $\bar{f}^k(0,0)=0$.

The upper bound value is:

$$UB(k)=\max_{\theta\le T}\bar{f}^k(T,\theta).$$

The algorithm complexity is $O(|\Gamma_t|^2 z)$, since we have $O(|\Gamma_t|^2)$ states (t,θ), and for each state we make $O(z)$ comparisons.

- *Combinatorial auction problem: Computational experiments*

In this subsection we discuss some computational experiments to evaluate the performance of the proposed algorithms H1, H2, H3 and H4, for the combinatorial auction problem. The comparison is made by comparing the heuristic solution values given by these algorithms, the upper bound given by the DPR, and the optimal solution value 'OPT' obtained by using ILOG OPL Studio 3.1.

We consider two sets of test problems. In '*Test Set* 1' we test the combinatorial auction algorithms on a set of instances where the number of bidders are $z=3,5$ or 10, and the number of (single-unit) goods are $T=25$, 50, 75 or 100, that is, the values of these parameters are similar to the number of projects and to the number of time slots we have considered for the experimentation of the whole multi-agent system model for the decentralized multi-project scheduling problem. In '*Test Set* 2' we consider test problems where both the number of

Table 10 Combinatorial auction results: *Test Set* 1

z	T	Min. HS	Ave. HS	Max. HS	Ave. OPT	Ave. gap%	Ave. DPR	Ave. H1	Ave. H2	Ave. H3	Ave. H4
3	25	0.45	1.12	1.91	1.12	0.009	1.18	0.68	0.72	1.02	1.08
3	50	0.27	1.03	2.08	1.03	0.010	1.25	0.86	0.87	1.02	1.03
3	75	0.40	1.00	1.90	1.00	0.012	1.20	0.79	0.84	0.95	1.00
3	100	0.59	1.21	1.81	1.21	0.008	1.34	0.85	0.88	1.19	1.21
5	25	0.81	1.19	2.03	1.19	0.010	1.33	1.06	1.03	1.18	1.17
5	50	0.63	1.33	1.93	1.35	1.070	1.59	1.24	1.28	1.26	1.29
5	75	1.04	1.63	2.56	1.65	1.190	2.08	1.48	1.56	1.50	1.61
5	100	1.02	1.96	3.14	1.98	1.290	2.21	1.63	1.70	1.87	1.96
10	25	0.88	1.64	2.40	1.69	2.920	2.02	1.44	1.47	1.54	1.52
10	50	1.44	2.10	3.52	2.17	2.690	2.75	1.97	1.92	2.05	1.97
10	75	1.63	2.63	3.71	2.63	0.050	3.45	2.55	2.52	2.51	2.55
10	100	1.36	2.82	3.88	2.86	1.160	3.68	2.76	2.71	2.76	2.72

Table 11 Combinatorial auction results: *Test Set* 2

z	T	Min. HS	Ave. HS	Max. HS	Ave. OPT	Ave. gap%	Ave. DPR	Ave. H1	Ave. H2	Ave. H3	Ave. H4
250	25	1.94	9.30	12.76	9.50	1.81	10.97	9.22	8.95	9.14	4.54
250	50	5.58	9.02	13.06	9.39	4.04	13.05	8.81	8.69	8.85	4.65
250	75	6.00	7.48	9.87	7.66	2.40	12.43	7.30	6.83	7.31	2.93
250	100	4.35	6.88	9.52	7.06	2.75	12.19	6.69	6.19	6.78	2.92
500	25	10.41	13.03	15.00	13.19	1.21	14.36	12.84	12.84	12.85	6.64
500	50	9.80	13.14	15.70	13.45	2.28	17.12	13.05	12.29	13.02	6.52
500	75	7.92	10.96	14.80	11.40	4.00	16.88	10.88	10.24	10.88	4.69
500	100	6.35	9.44	12.37	9.89	4.87	16.38	9.28	8.89	9.26	3.81
750	25	14.46	15.93	17.82	16.02	0.55	16.63	15.80	15.73	15.80	7.54
750	50	12.55	15.86	20.00	16.38	3.30	20.30	15.79	15.11	15.77	7.47
750	75	10.34	14.28	17.85	14.91	4.32	20.72	14.14	13.69	14.05	6.27
750	100	10.12	13.06	16.91	13.58	3.92	20.42	12.85	12.27	12.89	5.78
1000	25	15.54	17.29	19.21	17.68	2.15	17.98	17.16	17.04	17.14	7.55
1000	50	15.97	18.85	22.12	19.21	1.86	22.96	18.68	18.10	18.70	7.35
1000	75	13.09	18.34	22.58	18.84	2.80	25.12	18.08	17.59	18.14	8.42
1000	100	14.59	16.38	20.15	17.14	4.42	24.65	16.30	15.63	16.27	7.41

bidders and the number of goods are much more larger; in particular, we consider $z = 250$, 500, 750 or 1000 bidders, and $T = 25$, 50, 75, or 100 goods. All the problem instances are generated by using a random generator, as the one proposed by Sandholm et al. (2002) for the single-unit auctions; for each bidder, we have generated the bid by picking randomly a number of goods in the interval $[1, T]$; the bid prices are randomly generated according to the uniform distribution in the interval [0, 1]. Finally, for each (z, T) parameter combination, we have randomly generated 20 different instances.

Computational results are given in Tables 10 and 11 for '*Test Set* 1' and '*Test Set* 2', respectively. The tables are organized as follows. Columns one and two list the values of z and T, respectively; columns three, four and five report the minimum, the average and the maximum solution values of the best heuristic solution value 'HS' (i.e., the maximum among

the solution values provided by algorithms H1, H2, H3 and H4, respectively). Column six lists the average optimal solution value 'Ave. OPT'. Column seven reports the average gap (in percentage) 'Ave. gap %' between the optimal solution value 'OPT' and the best heuristic solution value 'HS' (i.e., $100 \times (\text{OPT} - \text{HS})/\text{OPT}$). Column eight lists the average solution value 'Ave. DPR' provided by the DPR. Finally columns nine to twelve list the average solution values provided by algorithms H1, H2, H3 and H4, respectively.

As one can see from Table 10, the average relative gap (in percentage) for all the bidders-goods combinations of '*Test Set* 1', is at most 2.92%, and smaller than 0,01% for $z = 3$. For the instances of '*Test Set* 2' (see Table 11) the average relative gap is at most 4.87%.

Acknowledgments The authors are grateful to the anonymous referees for their valuable comments and suggestions which contributed to improving the quality of the paper.

References

Agnetis, A., P.B. Mirchandani, D. Pacciarelli, and A. Pacifici. (2004). "Scheduling Problems with Two Competing Agents." *Operations Research*, 52(2), 229–242.

Arbib, C. and F. Rossi. (2000). "Optimal Resource Assignment Through Negotiation in a Multi-agent Manufacturing System." *IIE Transactions*, 32, 963–974.

Brucker, P., A. Drexl, R. Moehring, K. Neumann, and E. Pesch. (1999). "Resource-Constrained Project Scheduling: Notation, Classification, Models and Methods." *European Journal of Operational Research*, 112, 3–41.

Jennings, N.R. and M. Wooldridge. (1995). "Applying Agent Technology." *Applied Artificial Intelligence*, 9, 357–369.

Klemperer, P. (1999). "Auction Theory: A Guide to the Literature." *Journal of Economic Surveys*, 13, 227–286.

Knotts, G., M. Dror, and B.C. Hartman. (2000). "Agent-Based Project Scheduling." *IIE Transactions*, 32, 387–401.

Kolisch, R. (1996). "Serial and Parallel Resource-Constrained Project Scheduling Methods Revisited: Theory and Computation." *European Journal of Operational Research*, 90, 320–333.

Kolisch R. and A. Sprecher. (1996). "PSPLIB-A Project Scheduling Library." *European Journal of Operational Research*, 96, 205–216.

Kubiak, W., J. Błażewicz, P. Formanowicz, J. Breit, and G. Schmidt. (2002). "Two-Machine Flow Shops with Limited Machine Availability." *European Journal of Operational Research*, 136, 528–540.

Lova, A. and P. Tormos. (2001). "Analysis of Scheduling Schemes and Heuristic Rules Performance in Resource-Constrained Multiproject Scheduling." *Annals of Operations Research* 102, 263–286.

Lee, Y.-H., S.R. Kumara, and K. Chatterjee. (2003). "Multiagent Based Dynamic Resource Scheduling for Distributed Multiple Projects Using a Market Mechanism." *Journal of Intelligent Manufacturing*, 14, 471–484.

McAfee, R.P. and J. McMillan. (1987). "Auctions and Bidding." *Journal of Economic Literature*, 25, 699–738.

Parkes, D.C. (1999). "iBundle: An Efficient Ascending Price Bundle Auction." In *Proc. of first ACM Conference on Electronic Commerce*, pp. 148–157.

Rothkopf, M.H., A. Pereč, and R.M. Harstad. (1998). "Computationally Manageable Combinatorial Auctions." *Management Science*, 44, 1131–1147.

Sandholm, T. (2000). "Approaches to Winner Determination in Combinatorial Auctions." *Elsevier Decision Support System*, 28, 165–176.

Sandholm, T. and S. Suri. (2000). "Improved Algorithms for Optimal Winner Determination in Combinatorial Auctions and Generalizations." In *Proc. of National Conference on Artificial Intelligence*, pp. 90–97.

Sandholm, T., S. Suri, A. Gilpin, and D. Levine. (2002). "Winner Determination in Combinatorial Auction Generalizations." In *Proc. of the First International Conference on Autonomous Agents and Multi-Agent Systems*.

Wellman, M.P., W.E. Walsh, P.R. Wurman, and J.K. MacKie-Mason. (2001). "Auction Protocols for Decentralized Scheduling." *Games and Economic Behavior*, 35, 271–303.